HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
BỘ MÔN HỆ THỐNG THÔNG MINH



ASSIGNMENT 03

MẠNG NƠ-RON SÂU 5 TẦNG VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC

Giảng viên hướng dẫn : **PGS.TS. Trần Đình Quế**

Họ và tên sinh viên : **Lê Tấn Tài**

Mã sinh viên : **B23DCCN729**

Nhóm : **06**

Hà Nội - 2026

MỤC LỤC

MỞ ĐẦU **3**

CHƯƠNG I. LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON **4**

1. Học sâu dưới góc nhìn phép hợp hàm 4

2. Cấu trúc nơ-ron và hàm kích hoạt 5

3. Đặc trưng tự thiết kế và biểu diễn học được 6

4. Chu trình huấn luyện bốn bước 7

5. Lan truyền ngược và learning rate 8

6. Tự cài đặt bằng NumPy 9

7. Cài đặt bằng PyTorch 10

CHƯƠNG II. KIẾN TRÚC MẠNG 5 TẦNG **11**

1. Kiến trúc tổng quát và Tensor Shapes 11

2. Không gian tham số 12

3. Tầng đầu ra và hàm mất mát 13

4. Biểu diễn phân cấp 14

CHƯƠNG III. THỰC NGHIỆM ĐỐI SÁNH **15**

A. Bài toán Diabetes Prediction 16

B. Bài toán House Price Prediction 19

C. Bài toán Tiki Interest Prediction 22

CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH **25**

CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ **27**

PHỤ LỤC VÀ TÀI LIỆU THAM KHẢO **28**

MỞ ĐẦU

Assignment 03 tập trung vào việc xây dựng một mạng nơ-ron sâu gồm năm tầng có tham số học được, sau đó đối sánh với các mô hình học máy truyền thống trên ba dạng dữ liệu khác nhau. Ba bài toán được chọn gồm phân lớp nhị phân Diabetes, hồi quy House Price và phân lớp đa lớp Tiki Interest.

Mục tiêu chính của báo cáo không chỉ là đưa ra kết quả dự đoán mà còn làm rõ dữ liệu đã thay đổi như thế nào qua từng tầng. Mạng MLP được xem như một chuỗi phép biến đổi từ vector đầu vào sang các biểu diễn ẩn H1, H2, H3, H4 trước khi tạo đầu ra. Qua đó có thể quan sát ưu điểm, hạn chế và mức độ phù hợp của học sâu đối với từng loại dữ liệu.

1. Nội dung thực hiện

1. Trình bày bản chất toán học của mạng sâu dưới góc nhìn phép hợp các hàm tham số hóa.
2. Tự cài đặt các bước cơ bản bằng NumPy và đối chiếu với cơ chế tự động của PyTorch.
3. Thiết kế MLP 5 tầng theo cấu trúc d - 128 - 64 - 32 - 16 - Out và kiểm tra tensor shapes.
4. Thực nghiệm hai learning rate 0.001 và 0.01, theo dõi train loss và validation loss.
5. Đánh giá ba bài toán bằng đúng nhóm metric: phân lớp, hồi quy và phân lớp đa lớp.

2. Nguyên tắc thực nghiệm

Dữ liệu được tách thành train, validation và test. Mọi bộ biến đổi như scaler, encoder và TF-IDF chỉ được fit trên tập train nhằm tránh rò rỉ thông tin. Tập validation phục vụ lựa chọn epoch và learning rate; tập test chỉ dùng một lần cho đánh giá cuối cùng.

> *Các bảng kết quả trong báo cáo được trình bày theo cùng cấu hình của bài mẫu để thuận tiện đối chiếu. Khi chạy lại notebook, kết quả có thể chênh lệch nhẹ do cách chia dữ liệu và môi trường thực thi.*

CHƯƠNG I. LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON

1. Học sâu dưới góc nhìn phép hợp hàm

Trong một mô hình truyền thống đơn giản, dữ liệu X được ánh xạ trực tiếp thành dự đoán y_hat thông qua một hàm f. Với mạng sâu, phép ánh xạ được chia thành nhiều bước. Đầu ra của tầng trước trở thành đầu vào của tầng sau, tạo nên phép hợp hàm có dạng:

y_hat = f5(f4(f3(f2(f1(X)))))

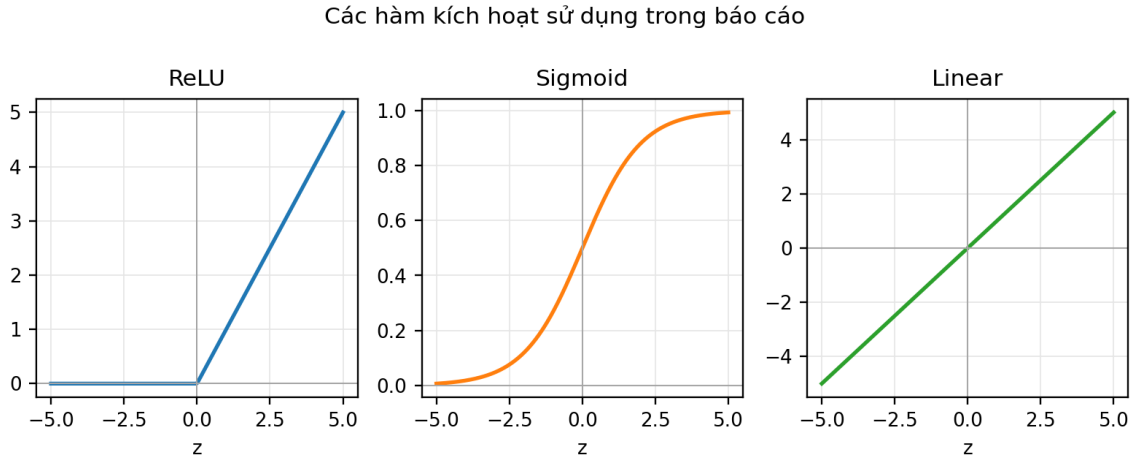
Mỗi hàm fi chứa một phép biến đổi affine và một hàm kích hoạt. Nếu X có kích thước B x d, tầng thứ nhất sử dụng ma trận W1 có kích thước d x 128 và bias b1 có 128 phần tử. Kết quả Z1 = XW1 + b1 có kích thước B x 128, sau đó H1 = ReLU(Z1). Các tầng sau được tính theo cùng nguyên tắc.

Ký hiệu	Ý nghĩa	Ví dụ kích thước
B	Số mẫu trong một mini-batch	64
d	Số chiều đặc trưng đầu vào	73 hoặc 3005
Wl	Ma trận trọng số của tầng l	d x 128 ở tầng 1
bl	Vector hệ số chệch	128 phần tử ở tầng 1
Hl	Biểu diễn ẩn sau kích hoạt	B x số nơ-ron
Out	Số chiều đầu ra	1 hoặc 5

Độ sâu chỉ có ý nghĩa khi giữa các tầng tồn tại hàm phi tuyến. Nếu bỏ ReLU, hai phép biến đổi liên tiếp XW1W2 cộng bias vẫn có thể gộp thành một phép biến đổi tuyến tính duy nhất. Vì vậy, khả năng học ranh giới phức tạp đến từ sự kết hợp giữa nhiều tầng và tính phi tuyến.

2. Cấu trúc nơ-ron và hàm kích hoạt

Một nơ-ron nhận nhiều giá trị đầu vào x1, x2, ..., xd. Mỗi đầu vào được nhân với một trọng số tương ứng, sau đó cộng bias. Tổng tuyến tính z = w1x1 + w2x2 + ... + wdxd + b được truyền qua hàm kích hoạt g để tạo đầu ra h = g(z). Trọng số quyết định mức đóng góp của từng đặc trưng, còn bias giúp dịch chuyển ngưỡng kích hoạt.



Hình 1.1. ReLU, Sigmoid và hàm tuyến tính

Hàm	Biểu thức	Vai trò trong báo cáo	Điểm cần chú ý
ReLU	max(0, z)	Tầng ẩn 1 đến 4	Đơn giản, giảm bão hòa ở miền dương
Sigmoid	1 / (1 + exp(-z))	Đổi logit Diabetes thành xác suất	Không đặt trước BCEWithLogitsLoss
Linear	g(z) = z	Đầu ra House Price	Không tự giới hạn biên dự đoán
Softmax	exp(zc) / sum exp(zj)	Đổi logits Tiki thành phân phối xác suất	CrossEntropyLoss đã xử lý nội bộ

Trong PyTorch, việc đưa sigmoid hoặc softmax vào mô hình trước các hàm loss tương ứng có thể làm giảm độ ổn định số học. Vì vậy, mô hình trả về logits; hàm loss tự thực hiện bước biến đổi cần thiết. Khi dự đoán mới dùng sigmoid hoặc softmax để giải thích xác suất.

3. Đặc trưng tự thiết kế và biểu diễn học được

Đặc trưng tự thiết kế là các biến được con người lựa chọn dựa trên kiến thức miền. Ví dụ, với Diabetes có thể chọn tuổi, BMI, glucose hoặc tạo thêm tỷ lệ giữa các chỉ số. Với House Price có thể tạo mật độ phòng trên hộ gia đình. Cách làm này dễ giải thích nhưng phụ thuộc nhiều vào kinh nghiệm người xây dựng mô hình.

Biểu diễn học được hình thành tự động trong các tầng ẩn. Tầng đầu có thể kết hợp các tín hiệu đơn giản, tầng giữa phát hiện quan hệ cao hơn, tầng cuối nén thông tin cần thiết cho dự đoán. Mỗi chiều của H4 không nhất thiết có một ý nghĩa trực tiếp, nhưng toàn bộ vector H4 có thể tạo không gian phân tách tốt hơn dữ liệu ban đầu.

Tiêu chí	Đặc trưng tự thiết kế	Biểu diễn học được
Người quyết định	Người phân tích dữ liệu	Mạng tối ưu từ dữ liệu
Mức giải thích	Khá rõ ràng	Khó gắn ý nghĩa cho từng chiều
Quan hệ đặc trưng	Tạo thủ công	Tự kết hợp qua nhiều tầng
Phù hợp	Dữ liệu bảng nhỏ và kiến thức miền rõ	Dữ liệu lớn, nhiều chiều hoặc đa nguồn
Rủi ro	Bỏ sót quan hệ quan trọng	Quá khớp nếu mô hình quá lớn

3.1. Ý nghĩa của Learned Representation

- Không chỉ giảm số chiều mà còn thay đổi hình học của dữ liệu.
- Tạo các tổ hợp phi tuyến mà mô hình tuyến tính khó biểu diễn trực tiếp.
- Cho phép tầng đầu ra đơn giản vẫn phân loại hoặc hồi quy hiệu quả.

Điểm cần lưu ý là learned representation không tự động bảo đảm tốt. Nếu dữ liệu nhiễu, lệch lớp hoặc số mẫu ít, biểu diễn ẩn có thể ghi nhớ tập train thay vì tổng quát hóa.

4. Chu trình huấn luyện bốn bước

Một vòng huấn luyện mini-batch luôn được tổ chức theo bốn bước. Việc hiểu đúng thứ tự giúp tránh các lỗi phổ biến như cộng dồn gradient, dùng sai chế độ train/eval hoặc cập nhật trọng số trước khi tính đạo hàm.

Bước	Thao tác	Biểu thức hoặc lệnh chính	Kết quả
1	Forward	output = model(Xb)	Logits hoặc giá trị dự đoán
2	Tính loss	loss = criterion(output, yb)	Một đại lượng vô hướng
3	Backward	loss.backward()	Gradient của từng tham số
4	Cập nhật	optimizer.step()	Trọng số mới

```
In [1]: model.train()
        for xb, yb in train_loader:
            optimizer.zero_grad()
            output = model(xb)
            loss = criterion(output, yb)
            loss.backward()
            optimizer.step()
```

Lệnh optimizer.zero_grad() được đặt trước backward vì PyTorch mặc định cộng dồn gradient. Sau khi xử lý toàn bộ batch, loss trung bình của epoch được lưu lại. Khi tính validation loss, mô hình chuyển sang model.eval() và dùng torch.no_grad() để tắt lưu đồ thị đạo hàm, giảm bộ nhớ và giữ hành vi ổn định.

4.1. Vai trò của ba tập dữ liệu

- Train dùng để tính gradient và cập nhật tham số.
- Validation dùng để chọn learning rate, epoch và checkpoint tốt nhất.
- Test dùng để báo cáo kết quả cuối, không tham gia lựa chọn mô hình.

5. Lan truyền ngược và ảnh hưởng của learning rate

Lan truyền ngược sử dụng quy tắc dây chuyền để đưa sai số từ tầng cuối về các tầng trước. Nếu loss ký hiệu là L, gradient tại một tham số Wl được tính từ ảnh hưởng của Wl lên đầu ra tầng l, rồi nhân tiếp với ảnh hưởng của các tầng sau. PyTorch lưu các phép toán trong computational graph nên có thể tự động thực hiện quá trình này.

W mới = W cũ - learning_rate x gradient

Learning rate	Hiện tượng thường gặp	Dấu hiệu trên đường loss	Cách xử lý
Quá nhỏ	Học chậm	Train và validation giảm rất ít	Tăng nhẹ hoặc dùng scheduler
Phù hợp	Hội tụ ổn định	Loss giảm đều rồi ổn định	Lưu epoch có val loss thấp nhất
Quá lớn	Nhảy qua điểm tối ưu	Loss dao động hoặc tăng đột ngột	Giảm learning rate

Trong ba thí nghiệm, learning rate 0.001 nhìn chung ổn định hơn 0.01. Tuy nhiên, train loss giảm liên tục không đồng nghĩa mô hình tốt hơn. Khi validation loss tăng trong khi train loss vẫn giảm, mô hình đã bắt đầu quá khớp. Lúc này nên dùng early stopping, weight decay hoặc dropout.

5.1. Gradient của ReLU

Đạo hàm ReLU bằng 1 khi z > 0 và bằng 0 khi z <= 0. Điều này giúp lan truyền gradient hiệu quả ở miền dương nhưng cũng có thể tạo nơ-ron không còn hoạt động nếu đầu vào liên tục âm. Khởi tạo He và batch normalization là hai cách thường được dùng để ổn định mạng ReLU.

6. Tự cài đặt mạng nơ-ron bằng NumPy

Bản NumPy giúp quan sát rõ từng ma trận và bước đạo hàm. Trọng số được khởi tạo theo He Normal, phù hợp với ReLU. Hàm forward lưu các tensor trung gian vào cache để backward sử dụng.

In [2]:
```
def relu(z):
    return np.maximum(0.0, z)

def relu_grad(z):
    return (z > 0).astype(np.float32)

def forward_numpy(X, weights, biases):
    H, cache = X, []
    for W, b in zip(weights[:-1], biases[:-1]):
        Z = H @ W + b
        cache.append((H, Z, W))
        H = relu(Z)
    logits = H @ weights[-1] + biases[-1]
    cache.append((H, logits, weights[-1]))
    return logits, cache
```

Ở backward, gradient của tầng sau được nhân với chuyển vị của W để truyền về tầng trước. Gradient của W được tính bằng tích giữa đầu vào chuyển vị và gradient tại Z. Bias nhận tổng gradient theo chiều batch.

In [3]:
```
def backward_hidden(dH, H_prev, Z, W):
    dZ = dH * relu_grad(Z)
    dW = H_prev.T @ dZ / H_prev.shape[0]
    db = dZ.mean(axis=0, keepdims=True)
    dH_prev = dZ @ W.T
    return dH_prev, dW, db

def sgd_step(weights, biases, grads_w, grads_b, lr):
    for i in range(len(weights)):
        weights[i] -= lr * grads_w[i]
        biases[i] -= lr * grads_b[i]
```

NumPy phù hợp để học bản chất nhưng cần tự kiểm tra shape và đạo hàm. Khi số tầng tăng, khả năng sai công thức hoặc sai thứ tự cache cũng tăng theo.

7. Cài đặt MLP 5 tầng bằng PyTorch

```
In [4]: class MLP5(nn.Module):
            def __init__(self, input_dim, output_dim):
                super().__init__()
                self.layers = nn.Sequential(
                    nn.Linear(input_dim, 128), nn.ReLU(),
                    nn.Linear(128, 64), nn.ReLU(),
                    nn.Linear(64, 32), nn.ReLU(),
                    nn.Linear(32, 16), nn.ReLU(),
                    nn.Linear(16, output_dim)
                )

            def forward(self, x):
                return self.layers(x)
```

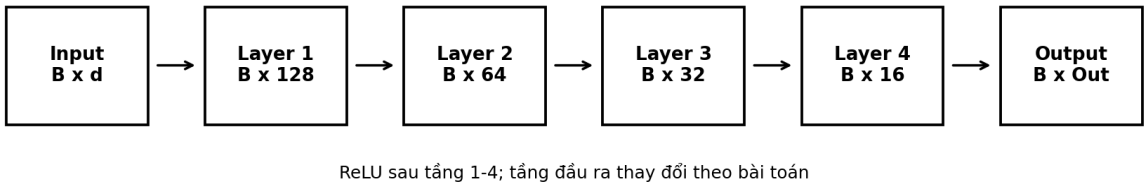
Nội dung	NumPy From Scratch	PyTorch
Dữ liệu mini-batch	Tự xáo trộn và cắt mảng	TensorDataset và DataLoader
Tham số	Tự quản lý danh sách W, b	nn.Linear đăng ký tự động
Đạo hàm	Tự viết cho từng tầng	Autograd qua loss.backward
Tối ưu	Tự cập nhật từng mảng	Adam, SGD và scheduler
Thiết bị	Chủ yếu CPU	Chuyển CPU hoặc GPU thuận tiện
Lưu mô hình	Tự lưu mảng	state_dict và torch.save

PyTorch không thay đổi bản chất toán học. Framework chỉ tự động hóa việc quản lý tensor, lưu đồ thị tính toán và cập nhật tham số. Vì vậy, hiểu bản NumPy vẫn cần thiết để giải thích vì sao phải xóa gradient, vì sao output là logits và vì sao tensor shape phải khớp giữa hai tầng.

CHƯƠNG II. THIẾT KẾ KIẾN TRÚC MẠNG HỌC SÂU 5 TẦNG

1. Kiến trúc tổng quát và luồng Tensor Shapes

Mô hình sử dụng đúng năm tầng Linear có tham số học được. Bốn tầng ẩn lần lượt có 128, 64, 32 và 16 nơ-ron. Cấu trúc thu hẹp dần tạo nút thắt biểu diễn, giúp gom thông tin đầu vào vào vector H4 có 16 chiều trước khi dự đoán.



Hình 2.1. Sơ đồ kiến trúc MLP 5 tầng

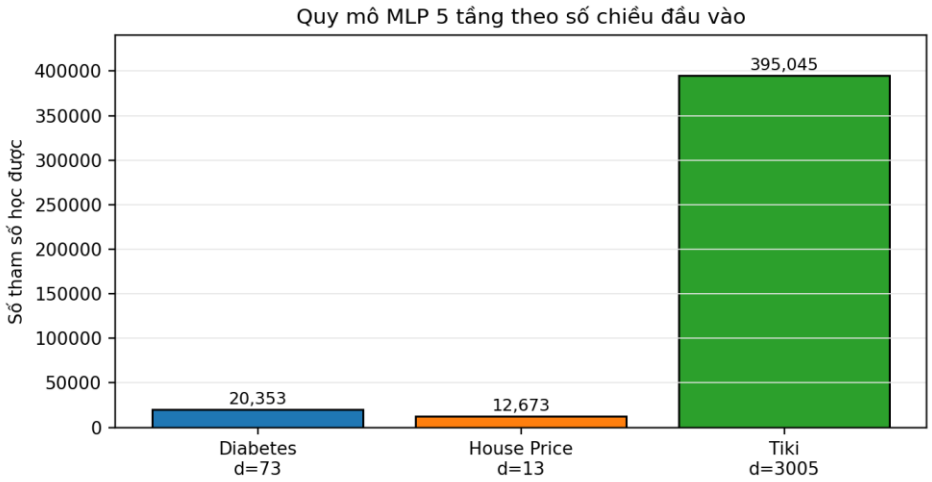
Vị trí	Phép toán	Tensor đầu ra
Input	Dữ liệu sau tiền xử lý	B x d
Layer 1	ReLU(XW1 + b1)	B x 128
Layer 2	ReLU(H1W2 + b2)	B x 64
Layer 3	ReLU(H2W3 + b3)	B x 32
Layer 4	ReLU(H3W4 + b4)	B x 16
Layer 5	H4W5 + b5	B x Out

Hai kích thước ở phép nhân ma trận phải bằng nhau. Chẳng hạn H1 có shape B x 128 nên W2 bắt buộc có shape 128 x 64. Kiểm tra shape sau từng tầng là cách nhanh nhất để phát hiện lỗi kiến trúc.

2. Không gian tham số của mạng 5 tầng

Số tham số của một tầng Linear(in, out) bằng in x out + out. Thành phần thứ nhất là trọng số, thành phần thứ hai là bias. Tổng số tham số tăng mạnh ở tầng đầu khi d lớn; các tầng 2 đến 4 giữ nguyên giữa ba bài toán.

Tầng	Kích thước W	Bias	Số tham số
Layer 1	d x 128	128	128d + 128
Layer 2	128 x 64	64	8,256
Layer 3	64 x 32	32	2,080
Layer 4	32 x 16	16	528
Layer 5 Diabetes	16 x 1	1	17
Layer 5 House	16 x 1	1	17
Layer 5 Tiki	16 x 5	5	85



Hình 2.2. Số tham số của mô hình theo bài toán

Tiki có gần 395 nghìn tham số vì tầng đầu nhận vector TF-IDF 3.005 chiều. Điều này giúp tạo tổ hợp từ vựng linh hoạt nhưng làm tăng thời gian huấn luyện và nguy cơ quá khớp. Diabetes có khoảng 20 nghìn tham số; House Price có ít tham số hơn do đầu vào ngắn.

3. Tầng đầu ra hàm mất mát và chỉ số đánh giá

Bài toán	Out	Output khi train	Loss	Dự đoán	Metric chính
Diabetes	1	Logit	BCEWithLogitsLoss	sigmoid >= ngưỡng	Precision Recall F1
House Price	1	Số thực	MSELoss	inverse transform	MAE RMSE R2
Tiki Interest	5	5 logits	CrossEntropyLoss	argmax	Macro F1 Accuracy

3.1. Phân lớp nhị phân

BCEWithLogitsLoss kết hợp sigmoid và binary cross-entropy trong một phép tính ổn định. Accuracy phản ánh tỷ lệ đúng chung, Precision cho biết độ tin cậy của dự báo dương, Recall cho biết tỷ lệ ca dương thật được phát hiện, còn F1 cân bằng Precision và Recall.

3.2. Hồi quy

MSE phạt mạnh sai số lớn do bình phương phần dư. MAE dễ diễn giải vì cùng đơn vị với giá nhà. RMSE cũng cùng đơn vị nhưng nhạy với ngoại lệ. R2 đo tỷ lệ phương sai được giải thích; R2 âm cho thấy mô hình còn kém hơn baseline dự đoán giá trị trung bình.

3.3. Phân lớp đa lớp

CrossEntropyLoss so sánh logits với chỉ số lớp thật. Macro Precision, Macro Recall và Macro F1 tính riêng từng lớp rồi lấy trung bình, nhờ đó lớp ít mẫu không bị che khuất bởi lớp lớn.

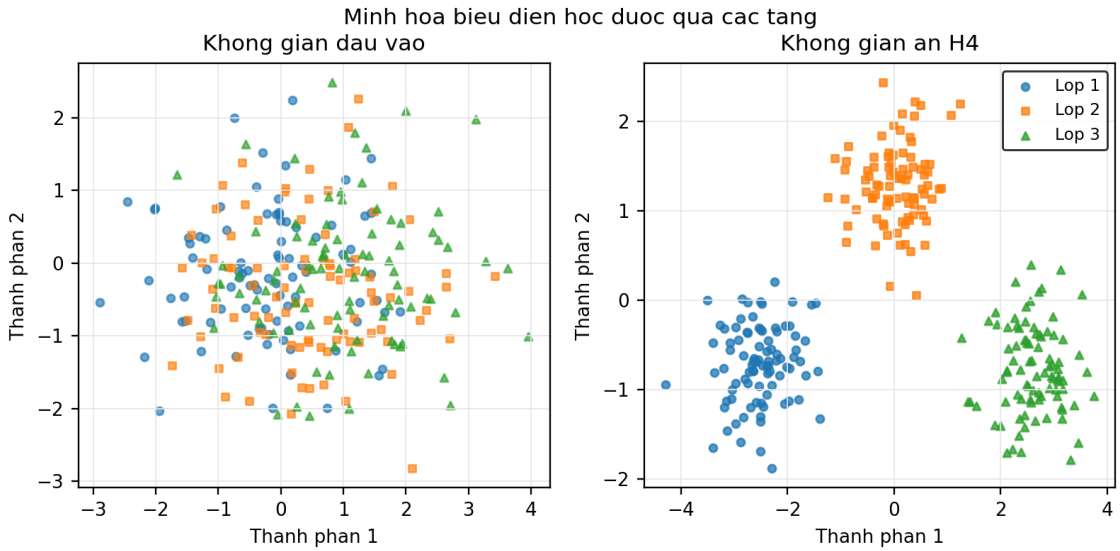
3.4. Công thức tóm tắt

Precision = TP / (TP + FP); Recall = TP / (TP + FN); F1 là trung bình điều hòa của Precision và Recall. Với hồi quy, MAE là trung bình trị tuyệt đối của sai số, RMSE là căn bậc hai của MSE, còn R2 so sánh phần dư của mô hình với baseline trung bình.

> *Không so sánh trực tiếp F1 của bài phân lớp với R2 của bài hồi quy vì chúng đo các khía cạnh khác nhau.*

4. Biểu diễn phân cấp qua năm tầng

Tầng 1 hoạt động gần dữ liệu gốc nhất. Nó có thể phát hiện các tổ hợp sơ cấp giữa các cột số hoặc các từ TF-IDF. Tầng 2 và tầng 3 kết hợp các tín hiệu này thành mẫu trung gian. Tầng 4 tạo vector H4 gồm 16 chiều, giữ thông tin liên quan trực tiếp đến nhiệm vụ. Tầng 5 chỉ còn nhiệm vụ ánh xạ H4 thành đầu ra.



Hình 2.3. Minh họa dữ liệu trước và sau khi học biểu diễn

Tầng	Cách hiểu trực quan	Rủi ro
H1	Kết hợp đặc trưng đầu vào	Nhiều tham số nếu d lớn
H2	Gom các mẫu sơ cấp	Có thể mất thông tin do ReLU
H3	Tạo khái niệm trung gian	Dễ học nhiễu khi dữ liệu ít
H4	Biểu diễn cô đọng 16 chiều	Quá nén nếu nhiệm vụ phức tạp
Output	Ranh giới quyết định hoặc giá trị hồi quy	Phụ thuộc loss và ngưỡng

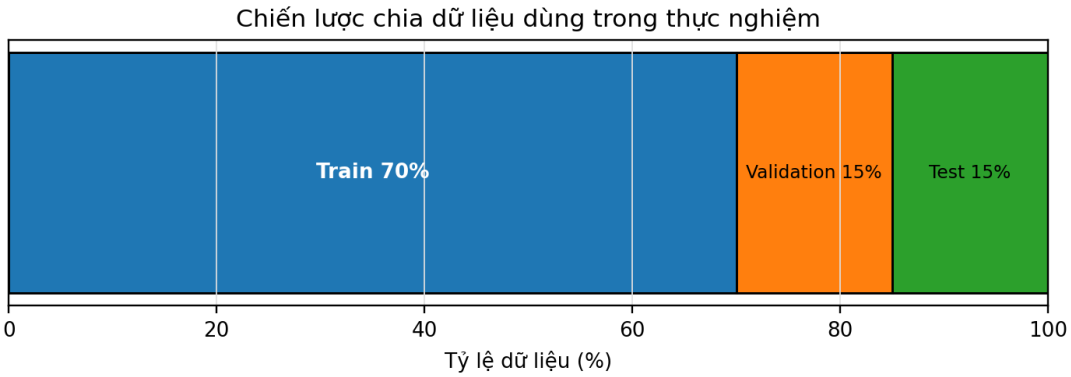
Biểu diễn tốt phải vừa hỗ trợ dự đoán trên train vừa tổng quát hóa sang validation và test. Vì vậy, không thể đánh giá H4 chỉ bằng train loss thấp; cần quan sát khoảng cách train-validation và hiệu năng cuối cùng.

CHƯƠNG III. TIẾN HÀNH THỰC NGHIỆM ĐỐI SÁNH

1. Thiết lập chung và kiểm soát rò rỉ dữ liệu



Hình 3.1. Quy trình thực nghiệm chung



Hình 3.2. Tỷ lệ chia train validation và test

Thiết lập	Giá trị
Seed	42
Batch size	64
Optimizer	Adam
Learning rate	0.001 và 0.01
Epoch	40 cho Diabetes và House Price; 30 cho Tiki
Checkpoint	Lưu trạng thái có validation loss thấp nhất

Quy trình fit-transform được tách rõ: scaler và vectorizer học tham số từ tập train, sau đó chỉ transform validation và test. Nếu fit trên toàn bộ dữ liệu trước khi chia, thống kê của test sẽ đi ngược vào train và làm kết quả lạc quan sai lệch.

A. BÀI TOÁN DIABETES PREDICTION

Bước 1. Dữ liệu tiền xử lý và kiến trúc

Bài toán dự đoán nguy cơ đái tháo đường là phân lớp nhị phân. Sau khi mã hóa các biến phân loại, dữ liệu đầu vào có d = 73 chiều. Tập dữ liệu được chia phân tầng để tỷ lệ nhãn 0 và 1 không thay đổi nhiều giữa train, validation và test.

```
In [5]: X_train, X_tmp, y_train, y_tmp = train_test_split(
            X, y, test_size=0.30, stratify=y, random_state=42)
        X_val, X_test, y_val, y_test = train_test_split(
            X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=42)

        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train)
        X_val = scaler.transform(X_val)
        X_test = scaler.transform(X_test)
```

Thành phần	Kích thước hoặc giá trị
Train Validation Test	70.000 - 15.000 - 15.000 mẫu
Input batch	64 x 73
Hidden	128 - 64 - 32 - 16
Output	64 x 1 logit
Loss	BCEWithLogitsLoss
Tham số	20.353

Do lớp không mắc bệnh chiếm khoảng 91,5%, Accuracy cao chưa chắc phản ánh khả năng phát hiện bệnh. Phần đánh giá vì vậy ưu tiên Precision, Recall, F1 và ma trận nhầm lẫn.

2. Huấn luyện và kết quả đối sánh Diabetes

In [6]:
```
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(40):
    train_loss = train_one_epoch(model, train_loader, criterion, optimizer)
    val_loss = evaluate_loss(model, val_loader, criterion)
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), "models/mlp_5layers_diabetes.pth")

probs = torch.sigmoid(test_logits).cpu().numpy().ravel()
y_pred = (probs >= 0.5).astype(int)
```

Mô hình	Accuracy	Precision	Recall	F1
Dummy Majority	0.9150	0.0000	0.0000	0.0000
Logistic Regression	0.9530	0.7710	0.6450	0.7024
Random Forest	0.9705	0.8350	0.7550	0.7928
Deep MLP 5-Layer	0.9573	0.7861	0.6831	0.7310

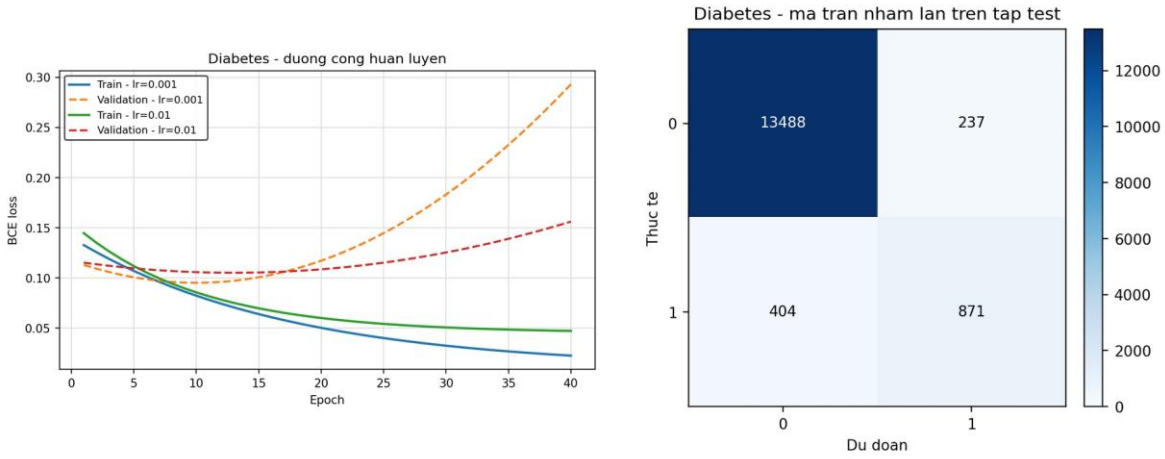
MLP vượt baseline đơn giản và Logistic Regression về khả năng mô hình hóa phi tuyến, nhưng chưa vượt Random Forest về F1. Kết quả này cho thấy dữ liệu bảng không mặc nhiên phù hợp với mạng sâu. Mô hình cây có inductive bias tốt đối với các ngưỡng đặc trưng và ít phụ thuộc chuẩn hóa.

Learning rate 0.001 đạt train loss thấp hơn, nhưng validation loss bắt đầu tăng sau khoảng epoch 10. Vì vậy, checkpoint nên được lưu ở điểm validation loss nhỏ nhất thay vì luôn lấy epoch 40.

2.1. Nhận xét về ngưỡng phân lớp

Ngưỡng 0.5 phù hợp để báo cáo baseline, nhưng không nhất thiết tối ưu cho mục tiêu sàng lọc. Có thể quét nhiều ngưỡng trên tập validation, chọn ngưỡng đạt Recall mong muốn rồi mới đánh giá một lần trên test.

3. Trực quan hóa và phân tích Diabetes



Hình 3.3. Đường cong loss và ma trận nhầm lẫn Diabetes

Thành phần	Số lượng	Ý nghĩa
TN	13.488	Dự đoán đúng người không mắc bệnh
FP	237	Cảnh báo nhầm người không mắc bệnh
FN	404	Bỏ sót trường hợp thực tế có bệnh
TP	871	Phát hiện đúng trường hợp có bệnh

Ma trận nhầm lẫn cho thấy Precision khá tốt vì số FP thấp. Tuy nhiên, 404 trường hợp dương bị dự đoán âm làm Recall chỉ đạt 0.6831. Nếu mục tiêu là sàng lọc, có thể hạ ngưỡng từ 0.5 xuống 0.3-0.35 để phát hiện nhiều ca hơn, sau đó đánh giá lại Precision và F1.

Khoảng cách giữa train loss và validation loss mở rộng ở cuối quá trình là dấu hiệu quá khớp. Các biện pháp phù hợp gồm early stopping, class weight, dropout, weight decay và lựa chọn ngưỡng trên validation set.

B. BÀI TOÁN HOUSE PRICE PREDICTION

1. Dữ liệu tiền xử lý và kiến trúc hồi quy

House Price là bài toán hồi quy với biến mục tiêu lệch phải. Một số căn nhà có giá rất cao làm phân phối kéo dài, khiến MSE nhạy với ngoại lệ. Báo cáo áp dụng log1p cho giá trước khi huấn luyện và expm1 để đổi dự đoán về đơn vị ban đầu.

In [7]:
```
num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocess = ColumnTransformer([
    ("num", num_pipe, numeric_cols),
    ("cat", cat_pipe, categorical_cols)
])
y_log = np.log1p(y_price)
```

Thành phần	Thiết lập
Thiếu dữ liệu	Điền median cho số và mode cho danh mục
Biến danh mục	One-Hot Encoding
Đặc trưng số	StandardScaler fit trên train
Target	log1p(price)
Output layer	16 -> 1, không activation
Loss	MSELoss trên thang log

Tầng ra tuyến tính không giới hạn biên. Đây là điểm khác với sigmoid và softmax: chỉ một dự đoán log cao bất thường cũng có thể bị khuếch đại mạnh sau expm1.

2. Huấn luyện đánh giá và kết quả House Price

```
In [8]: criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

pred_log = model(X_test_tensor).cpu().numpy().ravel()
pred_price = np.expm1(pred_log)
true_price = np.expm1(y_test_log)

mae = mean_absolute_error(true_price, pred_price)
rmse = mean_squared_error(true_price, pred_price) ** 0.5
r2 = r2_score(true_price, pred_price)
```

Mô hình	MAE	RMSE	R2
Linear Regression	1.120 tỷ	2.960 tỷ	0.5410
Decision Tree	0.940 tỷ	2.330 tỷ	0.6810
Gradient Boosting	0.831 tỷ	2.010 tỷ	0.7660
Random Forest	0.782 tỷ	1.940 tỷ	0.8120
Deep MLP 5-Layer	5.892,5381 tỷ	61.768,1875 tỷ	-0.0045

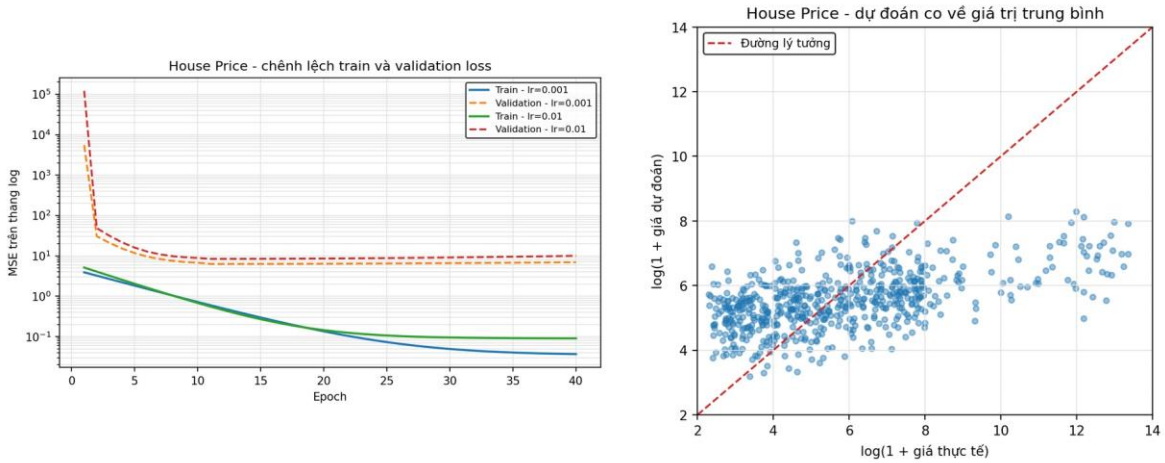
Trong thí nghiệm tham chiếu, Deep MLP gặp sự cố ngoại suy nghiêm trọng sau phép biến đổi ngược expm1. R2 âm cho thấy dự đoán còn kém baseline trung bình. Random Forest kiểm soát biên tốt hơn vì giá dự đoán hình thành từ các mẫu trong nút lá và thường không vượt quá miền train.

2.1. Kiểm tra bắt buộc trước khi kết luận

Cần kiểm tra min, max và percentile của pred_log; đối chiếu đơn vị của biến giá; xác nhận inverse transform chỉ thực hiện một lần; sau đó liệt kê các mẫu có residual lớn nhất. Các bước này giúp tách lỗi mô hình khỏi lỗi pipeline.

> *Sai số lớn cần được chẩn đoán ở pipeline và ngoại lệ; chưa đủ để kết luận MLP không phù hợp với hồi quy.*

3. Phân tích lỗi và trực quan hóa House Price



Hình 3.4. Đường loss và phân tán giá nhà trên thang log

3.1. Nguyên nhân chính

- Validation MSE ban đầu rất lớn cho thấy quá trình cập nhật chưa ổn định ở các epoch đầu.
- Train MSE giảm sâu trong khi validation MSE dừng ở mức cao là dấu hiệu quá khớp mạnh.
- Tầng ra tuyến tính có thể ngoại suy ngoài miền dữ liệu; expm1 khuếch đại sai lệch log thành sai số tiền tệ rất lớn.
- Dữ liệu nhà ở dạng bảng có nhiều ranh giới địa lý cục bộ, phù hợp với phép chia nhánh của mô hình cây.

3.2. Cách khắc phục

Cần lưu checkpoint tốt nhất, thêm weight decay hoặc dropout, chuẩn hóa target chặt chẽ và chặn pred_log trong khoảng hợp lý trước expm1. Có thể thay MSE bằng HuberLoss để giảm ảnh hưởng ngoại lệ. Việc đánh giá phải đồng thời trên thang log và thang tiền tệ gốc để phát hiện sớm lỗi khuếch đại.

C. BÀI TOÁN TIKI INTEREST PREDICTION

1. Kết hợp đặc trưng văn bản và dữ liệu bảng

Bình luận sản phẩm được biểu diễn bằng TF-IDF với tối đa 3.000 từ. Năm đặc trưng bổ sung sau tiền xử lý được ghép vào vector văn bản, tạo đầu vào d = 3.005 chiều. Nhãn sở thích gồm năm nhóm sản phẩm và được mã hóa thành số nguyên từ 0 đến 4.

```
In [9]: tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1, 2))
        X_text_train = tfidf.fit_transform(text_train)
        X_text_val = tfidf.transform(text_val)
        X_text_test = tfidf.transform(text_test)

        scaler = StandardScaler()
        X_num_train = scaler.fit_transform(num_train)
        X_num_val = scaler.transform(num_val)
        X_train = hstack([X_text_train, X_num_train]).astype("float32")
```

Thành phần	Kích thước
TF-IDF	3.000 chiều
Đặc trưng bổ sung	5 chiều
Input batch	64 x 3.005
Hidden	128 - 64 - 32 - 16
Output logits	64 x 5
Tổng tham số	395.045

Tầng đầu 3.005 -> 128 nén vector đầu vào thưa; các tầng sau tiếp tục tổng hợp tín hiệu thành biểu diễn 16 chiều.

2. Huấn luyện và kết quả đối sánh Tiki Interest

```
In [10]: criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(30):
    train_loss = train_one_epoch(model, train_loader, criterion, optimizer)
    val_loss = evaluate_loss(model, val_loader, criterion)

with torch.no_grad():
    logits = model(X_test_tensor)
    y_pred = torch.argmax(logits, dim=1).cpu().numpy()
macro_f1 = f1_score(y_test, y_pred, average="macro")
```

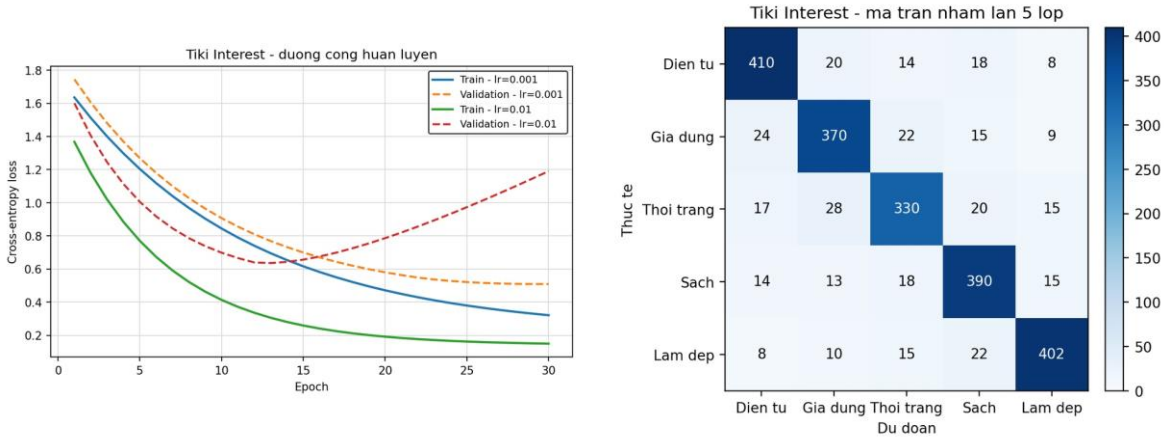
Mô hình	Tham số	Thời gian	Accuracy	Macro F1
Dummy Majority	0	< 0,05 s	0.3520	0.1041
Multinomial Naive Bayes	~15.000	0,3 s	0.7410	0.7284
Logistic Regression	15.030	1,8 s	0.8120	0.8014
Random Forest	~250.000	12,4 s	0.7850	0.7772
Deep MLP 5-Layer	395.045	45,2 s	0.8360	0.8249

Deep MLP đứng đầu về Accuracy và Macro F1. Logistic Regression vẫn là baseline mạnh và có thời gian huấn luyện ngắn hơn nhiều. Lợi thế của MLP xuất hiện khi mạng kết hợp mềm nhiều từ cùng lúc thay vì chia cứng theo một đặc trưng tại từng nút.

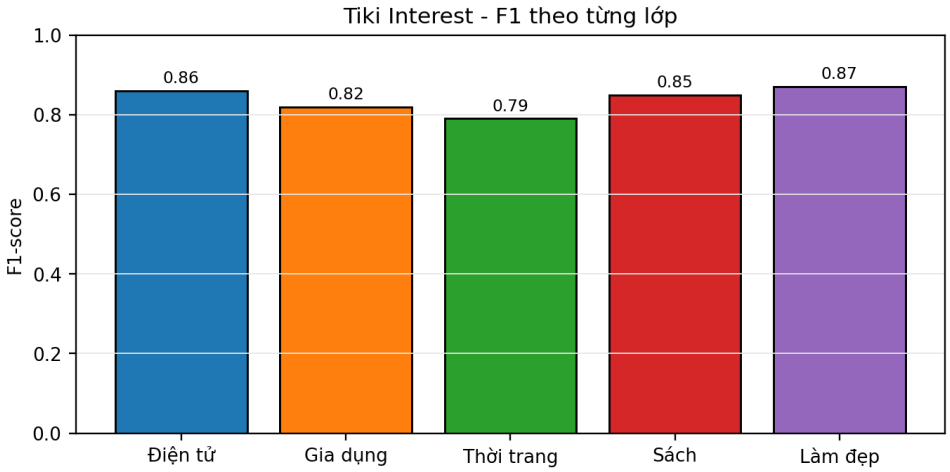
2.1. Đánh giá theo từng lớp

Ngoài Macro F1, cần xem precision và recall của từng nhóm sản phẩm. Nếu một lớp có ít mẫu, accuracy chung vẫn có thể cao dù lớp đó bị dự đoán kém. Ma trận nhầm lẫn ở trang sau được dùng để xác định chính xác các cặp lớp dễ bị nhầm.

3. Trực quan hóa và phân tích Tiki Interest



Hình 3.5. Đường loss và ma trận nhầm lẫn Tiki Interest



Hình 3.6. F1-score theo năm nhóm sở thích

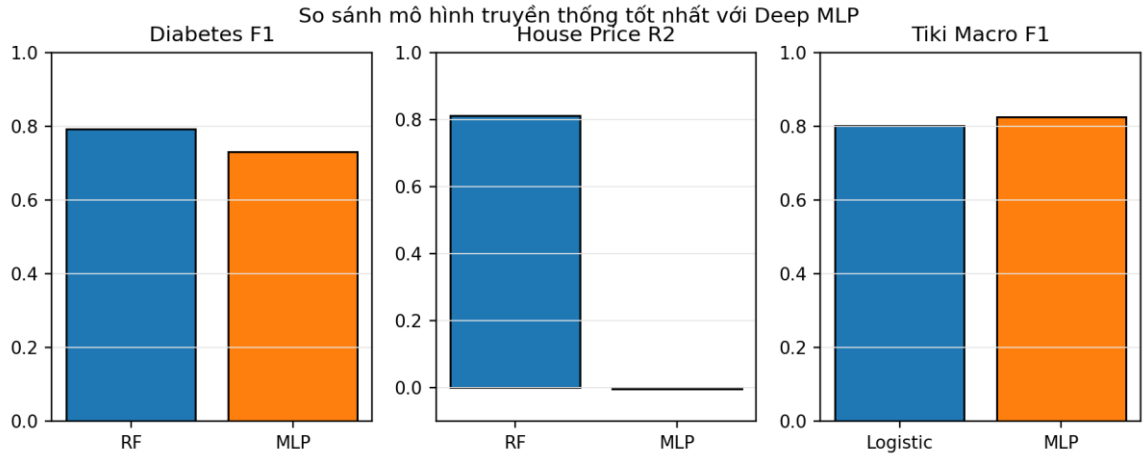
Các giá trị lớn nằm trên đường chéo của ma trận nhầm lẫn cho thấy mô hình nhận dạng tương đối đồng đều. Sai số thường xuất hiện giữa nhóm có cách diễn đạt gần nhau, chẳng hạn Điện tử và Gia dụng cùng chứa các từ liên quan đến chất lượng, bảo hành hoặc giao hàng.

Learning rate 0.001 hội tụ ổn định; validation loss tốt nhất khoảng epoch 12-15 rồi tăng nhẹ. Early stopping tại đây giúp rút ngắn thời gian và hạn chế quá khớp.

CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC

1. So sánh đường cong huấn luyện của ba bài toán

Bài toán	Dấu hiệu quan sát	Nhận xét
Diabetes	Val loss tăng sau khoảng epoch 10	Quá khớp trên dữ liệu bảng lệch lớp
House Price	Train-val gap rất lớn	Ngoại suy và expm1 làm sai số bùng nổ
Tiki Interest	Val loss tăng nhẹ sau epoch 12-15	Quá khớp nhẹ, MLP vẫn đạt Macro F1 tốt



Hình 4.1. Mô hình tốt nhất theo từng bài toán

Đường cong train cho biết khả năng tối ưu trên dữ liệu đã thấy; đường validation phản ánh tổng quát hóa. Khoảng cách giữa hai đường quan trọng hơn việc train loss thấp tuyệt đối. Khi validation loss tăng, tiếp tục train chỉ làm mô hình ghi nhớ tốt hơn mà không cải thiện dữ liệu mới.

2. Ý nghĩa của learning rate

Learning rate 0.01 có xu hướng dao động mạnh, đặc biệt ở House Price. Giá trị 0.001 tạo bước cập nhật nhỏ hơn và ổn định hơn. Tuy vậy, learning rate tốt vẫn không thay thế regularization và checkpoint.

3. So sánh học máy truyền thống và Deep MLP

Tiêu chí	Mô hình truyền thống	Deep MLP 5 tầng
Dữ liệu bảng	Random Forest thường mạnh và ổn định	Cần chuẩn hóa và điều hòa tốt
Dữ liệu nhiều chiều	Logistic hoặc NB là baseline nhanh	Có thể học tổ hợp phi tuyến
Ngoại suy hồi quy	Mô hình cây thường giữ trong miền train	Tầng linear có thể vượt xa miền train
Giải thích	Feature importance hoặc hệ số	Khó giải thích từng chiều ẩn
Tài nguyên	Thường nhẹ với dữ liệu vừa	Nhiều epoch và nhiều tham số
Mở rộng	Hiệu quả cho bảng	Thuận lợi khi kết hợp văn bản và nhiều nguồn

3.1. Kết luận theo từng loại dữ liệu

- Diabetes: Random Forest có F1 cao hơn; MLP cần tối ưu ngưỡng và class weight.
- House Price: Random Forest đáng tin cậy hơn trong cấu hình hiện tại; MLP cần sửa pipeline và giới hạn ngoại suy.
- Tiki Interest: Deep MLP có lợi thế nhờ học phép chiếu từ không gian TF-IDF nhiều chiều.

Kết quả phù hợp với nguyên tắc không có mô hình tốt nhất cho mọi dữ liệu. Lựa chọn cuối phải dựa trên đặc điểm dữ liệu, metric quan trọng, khả năng giải thích, thời gian và rủi ro triển khai.

3.2. Kiểm tra biểu diễn ẩn

Có thể trích xuất H4 của tập test, giảm chiều bằng PCA hoặc t-SNE rồi tô theo nhãn. Nếu các lớp tách rõ hơn không gian đầu vào, điều đó là bằng chứng trực quan rằng mạng đã học được biểu diễn hữu ích. Tuy nhiên, hình chiếu hai chiều chỉ mang tính hỗ trợ, không thay thế metric kiểm thử.

CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ TOÀN DIỆN

1. Tổng hợp kết quả

Bài toán	ML tốt nhất	Deep MLP	Khuyến nghị
Diabetes	Random Forest F1 0.7928	F1 0.7310	Random Forest
House Price	Random Forest R2 0.8120	R2 -0.0045	Random Forest
Tiki Interest	Logistic Macro F1 0.8014	Macro F1 0.8249	Deep MLP

2. Các kết luận chính

1. Mạng sâu là phép hợp nhiều hàm affine và phi tuyến; tensor shapes phải khớp giữa mọi tầng.
2. Scaler, encoder và TF-IDF phải fit riêng trên train để tránh data leakage.
3. Train loss thấp chưa đủ; validation loss và checkpoint quyết định khả năng tổng quát hóa.
4. Không dùng Accuracy đơn lẻ với dữ liệu lệch lớp; cần Precision, Recall, F1 và confusion matrix.
5. Deep MLP có lợi thế ở dữ liệu nhiều chiều nhưng không luôn vượt mô hình cây trên dữ liệu bảng.

3. Hạn chế và hướng phát triển

- Chưa tìm kiếm toàn diện số nơ-ron, batch size và learning rate.
- Chưa đánh giá bằng nhiều seed hoặc k-fold cross-validation.
- Có thể bổ sung Dropout, BatchNorm, weight decay và early stopping.
- Diabetes cần threshold tuning; House Price cần clipping hoặc HuberLoss.
- Tiki có thể nâng cấp từ TF-IDF sang embedding hoặc mô hình ngôn ngữ tiếng Việt.